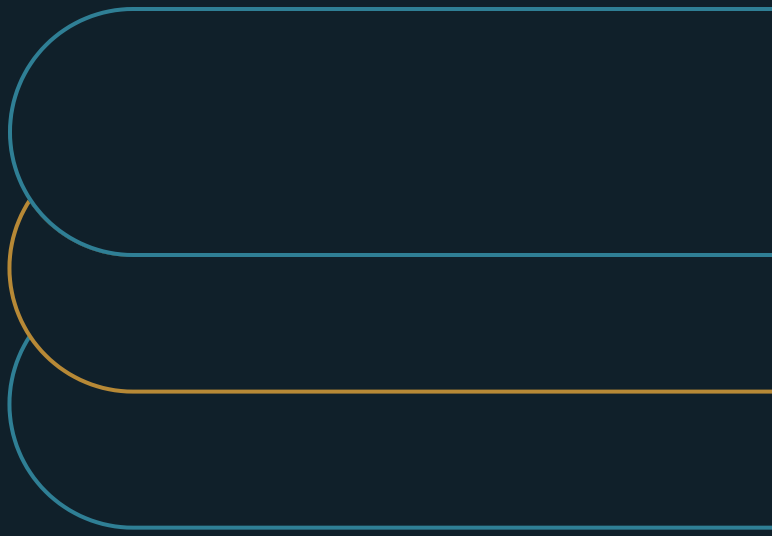
11

WHITE PAPER · AI OPERATING GOVERNANCE

An AI Evaluation Is a Production Operating Control

Govern changing workflow behavior, not a one-time test



Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Treat 'the system works' as a claim with a date on it - and keep the date current.

A model can pass an evaluation and still fail the operation that adopts it, because the test measured a moment and the operation lives in a stream. This paper turns evaluation from a launch gate into an operating control: a seven-discipline loop that tests real cases, monitors change, converts incidents into evidence, and keeps a named owner able to change what the workflow relies on.

01 Benchmarks, red teams, and production evaluation answer different questions. Stretching one to cover another authorizes reliance the evidence does not cover.

02 Write thresholds before seeing results, attached to case classes and weighted by consequence. Averages are where severe failures hide.

03 Run the staleness check: last evaluation evidence versus last material change. A gap means faith with paperwork.

The argument

Benchmarking, red teaming, and production evaluation answer different questions. Production evaluation is a living control. It tests true-to-life workflow cases, monitors change, converts incidents into new test cases, and changes what the workflow may rely on when the evidence changes. The shift it names is an old one in operations: from inspecting quality in at a final gate to controlling it continuously in the running process — the move from end-of-line inspection to statistical process control that manufacturing made a century ago, now owed by any workflow that leans on a model whose behavior drifts.

The failure pattern that teaches this difference tends to arrive months after a successful launch. A workflow clears its pre-launch evaluation convincingly: a proper case set, honest thresholds, results everyone can stand behind. Then a few small things happen that nobody connects. The vendor ships a model update. An upstream team changes how inputs are formatted. The business extends the workflow to a case type the test set never contained. Each change is individually announced, individually reasonable, and individually invisible to an evaluation that ran once, passed once, and retired to a folder. Quality degrades for weeks before a downstream complaint surfaces it. The launch evaluation was not wrong. It was abandoned.

That is the general condition. A model can pass an evaluation and still fail the operation that adopts it, because the test measured a moment and the operation lives in a stream. Cases drift from the tested population. Users rely on output in ways the design never assumed. Retrieval data ages. Vendors change

the engine underneath. None of this is exotic; it is what production means. Which is why evaluation cannot be a launch artifact — it has to be an operating control. Two forces guarantee the drift. Machine-learning systems accrue hidden maintenance debt as their data dependencies and the outside world shift, so yesterday's passing system is not today's [3]; and the generative layer can stay fluent while ceasing to be correct, producing plausible output a one-time test never anticipated [4].

This paper describes that control as a production evaluation loop with seven disciplines. NIST's generative-AI profile argues for systematic, continuing measurement. Microsoft's red-team lessons show how much failure-finding depends on deliberate, resourced effort [1][2]. The loop generalizes the machinery I have built around production AI workflows. It is written for AI product leaders, workflow owners, risk teams, and technology executives.

Three tests, three questions

The word "evaluation" covers three different jobs, and conflating them is the root error in most AI assurance conversations. A benchmark compares capability across models on standard tasks. It answers: which engine is stronger, in general? A red team searches for ways a system can be made to fail. It answers: where are the weaknesses a motivated actor or unlucky input could find? A production evaluation tests a specific workflow's cases under its own consequences. It answers: may this operation rely on this system, for these cases, right now?

Each test is necessary. None substitutes for the others. The characteristic misuse is stretching: a strong benchmark cited as proof a workflow is safe, a pre-launch red team cited as proof the system remains safe a year later, a passing workflow evaluation cited to authorize case types it never contained. Stretching feels efficient because tests are expensive. It is actually the most expensive move available, because the stretched result authorizes reliance the evidence does not cover — and the gap is exactly where the incidents live [2].

The discipline is documentary and cheap. Every test on record states which question it answers, what population it covers, what environment it ran in, and what decision it may support. When a governance decision needs an answer no existing test provides, commission the missing test. Never promote an existing result beyond what its own record claims.

The three-test separation also allocates scarce effort correctly. Benchmarks are largely free: published, maintained by others, useful for shortlisting engines. Red teams are episodic and expensive: commissioned at launch, after major change, and on a cadence matched to adversarial exposure [2]. Production evaluation is continuous and owned. It is the only one of the three the operation must run itself — because only the operation has its cases, its consequences, and its definition of fit. Budgets that treat the three as one line item reliably fund the first two and starve the third.

The production evaluation loop

The loop has seven disciplines. The first three build the evaluation's substance: cases, thresholds, and the separation above. The next three keep it alive: monitoring, change events, and incident learning. The last

connects it to power: someone must be able to change what the workflow relies on, or the apparatus is measurement without consequence.

EXHIBIT 1

The evidence-renewal loop

Representative cases

Real workflow history defines what the evidence actually covers.

Consequence thresholds

Case-class limits are written before results and weighted by failure cost.

Signals and change

Drift, overrides, exceptions, and material changes trigger review or regression.

Incident learning

Failures and near misses become maintained cases rather than isolated postmortems.

Reliance authority

A named owner narrows, pauses, or restores permitted use as evidence changes.

Build true-to-life workflow cases

The case set is the evaluation's claim to relevance. Built from convenience — the examples the build team had at hand, the demos that impressed — it tests the system's comfort zone and certifies nothing beyond it. Built true to life, it samples production history with the mess left in, weights the subgroups that matter, includes the exceptions that consume operating judgment, and adds the foreseeable misuse a red-team mindset contributes.

Provenance separates the two kinds of case set, and it should be written down as part of the set. Where did each case class come from? Why is its frequency in the set defensible against production reality? Which reference answers are certain, and which are judgment calls with recorded uncertainty? Domain experts belong in this work. The operation's people know which cases are hard and why, and their afternoon of taxonomy is worth more than a month of synthetic generation.

The set's boundary is a governance output in its own right. Permitted use extends to the population the cases represent, and no further. The expansion that breaks workflows — a new case type, admitted without a case-set extension — is precisely the move this discipline prohibits. New population, new cases, new results, then new reliance. The order is the control.

Size is the least important property of a case set, and the most argued about. Two hundred well-chosen cases with honest reference answers govern better than ten thousand scraped ones, because the failure

analysis that follows a run has to be read by humans who will act on it. Start small and true to life, and let the incident pipeline grow the set where reality says it is thin.

Define thresholds by consequence

A single accuracy number is an average, and averages are where severe failures hide. Ninety-six percent overall can contain a critical case class failing three times in ten — invisible in aggregate, intolerable in operation. So thresholds attach to case classes, weighted by what a failure in that class costs. Acceptance rates for routine classes. Stricter floors for consequential ones. Abstention and escalation expectations where the right answer is "route to a human." Latency bounds where timing is part of correctness.

Abstention deserves explicit thresholds because it is the workflow's pressure valve. A system that answers everything is running without one. A system that abstains too readily exports its work back to the humans it was meant to relieve. Set an expected abstention band per case class, and alarm on both edges. That catches two failures with one instrument: overconfidence on cases the system should route away, and silent capability collapse disguised as caution.

Write the thresholds before seeing results. Afterward, they negotiate themselves toward whatever the system achieved, and the evaluation becomes a mirror instead of a bar. Writing them first forces the uncomfortable, valuable conversation about what the operation actually requires: which failures are absorbable, which are expensive, and which are the reason the workflow has a human in it at all.

The decision rule keeps the aggregate honest: a critical-class failure pauses or narrows reliance even when the average looks excellent. Review load belongs in the threshold set too. False rejections that flood human reviewers are a real cost, and a threshold structure that ignores them will be quietly overridden by the operation defending its own capacity.

Monitor production signals

Between formal evaluations, the workflow itself broadcasts evidence — if anything is listening. Input distributions shift. Output patterns move. Reviewer overrides cluster. Exceptions rise in one case class. Users develop workarounds. Downstream corrections pile up. Most companies instrument uptime and cost and stop there. They can see the system running. They cannot see whether it still fits the work.

The monitoring discipline instruments fitness: input characteristics against the case set's assumptions, output shapes against history, review decisions and override reasons, exception rates by class, and sampled quality checks on live cases. None of this needs to be elaborate at the start. A monthly sample read by someone who knows the work, plus drift flags on the obvious distributions, catches most of what matters. Grow the instrumentation where the signals prove valuable — not where the tooling catalog suggests.

The design question is response, not collection. Each signal needs a threshold, and each threshold a named owner who acts when it trips: a focused evaluation on the moving class, a review-capacity check, a case-set extension — or, when a signal calls reliance itself into question, an escalation to the owner who can narrow or pause it. Signals collected without response thresholds become dashboard wallpaper — and

their real function becomes retrospective: explaining, after the incident, that the evidence had been available all along.

Reviewer disagreement is a monitoring signal in its own right, and an underused one. When two qualified reviewers sampling the same live cases diverge on what counts as acceptable, the workflow has either a drifting system or a drifting standard. Both need governance attention. Tracking reviewer agreement on the monthly sample costs almost nothing and catches quality-definition drift no automated check can see. This matters because human monitoring is itself fallible: sustained oversight of a mostly-reliable system breeds complacency, so a disagreement signal that forces a second look is worth more than an extra reviewer who waves the stream through [5].

Turn change into a test event

Production AI systems change constantly, and every change class can invalidate prior evidence: model updates, prompt edits, retrieval-source changes, tool additions, policy revisions, workflow redesigns, user-population shifts. The operating question is never whether to allow change. It is whether change reaches production through a path that re-tests what the change could have broken.

The discipline is regression scoped by change class. A vendor model update reruns the full case set. A prompt refinement reruns the affected classes. A new retrieval source reruns retrieval-dependent cases plus a contamination sample. A workflow redesign may require new cases entirely. Version history ties each production state to its evidence, and approval scales with the consequence of what the change touches. The unannounced upstream formatting change is the canonical miss: nobody owned the question "what does this change touch?"

The rule is symmetrical with the case-set boundary. Deployment waits where regression evidence is incomplete. Reliance narrows where a change degraded a class the operation depends on. Teams accept this discipline readily for code and resist it for prompts and models — though a prompt edit can move behavior more than most code releases.

Vendor changes deserve their own contract clause, negotiated before dependence: advance notice windows for model updates, version pinning where offered, and change logs specific enough to scope a regression. Providers vary widely on all three, and the variance is purchasing-relevant information. A vendor that cannot say what changed has made its customers' regression scope "everything" — a recurring cost that belongs in the price comparison next to the per-token rate.

Learn from incidents and near misses

When the workflow fails in production — or nearly does — the failure is a free case, paid for at incident prices. The wasteful response repairs the instance: fix the record, apologize downstream, adjust the one prompt. The evaluative response converts it: a short causal review across the usual suspects — model, data, interface, instructions, review capacity, permissions, operating pressure — then a new case class in the set, a threshold or monitoring adjustment if the failure revealed a blind spot, and recurrence tracking to verify the fix held.

Near misses deserve equal standing, and mostly do not get it. A failure caught by an alert reviewer is evidence that the control worked — and evidence that the system produced the failure. Both facts belong in the record, because the next instance may not meet the same alert reviewer on the same unhurried day. Programs that learn only from consequential incidents have chosen to learn at the maximum available price per lesson.

Over quarters, the incident-to-case pipeline becomes the evaluation's growth mechanism. The set stops being the launch team's best guess about hard cases and becomes the operation's accumulated memory of actual ones. That memory is a portfolio asset: new deployments in adjacent workflows inherit case classes their own launches would have discovered expensively.

That review's breadth is what makes it worth running, because the cause is frequently not the model at all. A misrouted case may trace to an upstream data change. An accepted bad answer may trace to a review queue running at twice its designed volume. Converting incidents into cases without that breadth just grows the case set while the actual defect recurs untouched.

Assign authority to change reliance

Everything above produces evidence. This discipline gives evidence somewhere to go. A named workflow owner holds the reliance decision: what the operation may depend on, for which cases. A technical owner holds the system and its versions. A risk voice holds challenge rights. A defined forum reviews results and signals on cadence, and an emergency route exists for findings that cannot wait.

The failure this prevents is the analyst-report pattern: evaluations run, results circulate, and nothing changes, because no one owns the connection between evidence and permission. The tell is a finding everyone read and no one acted on — a threshold breached in an appendix while adoption momentum carried the workflow forward. Where that tell appears, the company does not have an evaluation control. It has evaluation content.

The authority's evidence is decision-shaped. Reliance narrowed within days of a breached threshold. Restorations tied to re-tests. Emergency stops exercised at least once against a rehearsed fallback. A record connecting each change in permitted use to the evidence that drove it. Time from signal to decision is the metric that matters. Evidence that changes reliance in days is a control. Evidence that changes it in quarters is history.

Cadence completes the design: a standing review — monthly in most operations — that reads the monitoring summary, the change log, and the incident conversions, and issues one of three findings per workflow. Reliance confirmed. Reliance adjusted. Or evidence gap opened, with an owner and a date. A review that ends without one of the three has been a briefing.

The loop as a system

The disciplines compound in a specific way. True-to-life cases make thresholds meaningful. Thresholds make monitoring actionable. Monitoring makes change events detectable. Change discipline keeps the case evidence current. Incidents feed the case set its hardest members. And named authority makes all of it consequential. Remove any one and the others degrade predictably. Most commonly, companies build

cases and thresholds, skip monitoring and change discipline, and rediscover at incident time that their evidence describes a system that no longer exists.

The loop is also the connective tissue between the other controls an AI operation needs. Authority ladders decide how much the workflow may act on the system's output; the loop supplies the evidence those grants cite. Challenge protocols catch the specific bad answer; the loop catches the drift no single answer reveals. Reliance decisions at the production gate cite the launch evaluation; the loop is what keeps that citation truthful afterward. One evaluation, maintained, serves all three.

Standing the loop up

The loop installs in the order its evidence compounds:

- Create the first case set from real workflow history, not a generic benchmark.
- Write acceptance and stop thresholds before running the evaluation.
- Connect production telemetry and reviewer decisions to a maintained evaluation backlog.
- Run regression after material change, and a scheduled review even when no incident occurs.

The first loop cycle doubles as archaeology. Building the case set from history surfaces how the workflow actually behaves: the exception classes nobody had counted, the case types that dominate reviewer time, the quiet workarounds users built. Teams routinely find that the evaluation's first contribution preceded any test run — assembling true-to-life cases taught them what the workflow was.

Resourcing is the conversation to have at installation, because the loop is standing work: case-set maintenance, monitoring review, regression runs, incident conversion. The sizing rule is matching effort to reliance. A workflow the operation depends on daily deserves a few hours of evaluation stewardship weekly. A workflow that cannot justify that stewardship is implicitly declaring how much it should be relied on. Unfunded evaluation does not degrade gracefully. It stops silently, while its last passing result keeps circulating as if current.

Ownership placement matters less than ownership reality. The stewardship can sit with the product team, a platform group, or a quality function — provided the reliance authority stays with the workflow owner and the challenge right stays independent of the team whose system is being judged. The anti-pattern is evaluation owned entirely by the builders. Not because builders are dishonest, but because the evaluation's job includes disappointing them.

What the record should show

A living evaluation leaves a current trail:

- A case set with provenance, subgroup coverage, and a stated population boundary.
- Thresholds by consequence class, written before results, with review load included.
- Monitoring signals with response thresholds — and the responses they triggered.
- Change events matched to regression runs and version-tied evidence.

- Incidents and near misses converted into case classes, with recurrence tracked.
- Reliance changes — narrowings, pauses, restorations — tied to evidence, with dates.

Auditors and enterprise buyers increasingly ask for exactly this trail, which gives the loop a second constituency beyond the operation itself. A company that can produce current, versioned, risk-weighted evaluation evidence answers due-diligence questionnaires from its records. One that cannot answers them with meetings. The difference is measured in sales-cycle weeks and audit findings — which is often the argument that finally funds the stewardship.

The staleness check is the cheapest audit available in this field. Compare the date of the last evaluation evidence with the date of the last material change to the system, the data, or the workflow. A gap means the operation is running on faith with paperwork. The loop's purpose is to make that gap structurally impossible: every change either ran through the evidence machinery, or it is visibly outstanding on a named owner's list.

The strongest counterargument

No evaluation can prove universal safety or future performance. It provides bounded evidence for a named use under observed conditions. Honest limits, maintained cases, and authority to change reliance are worth more than a large one-time score.

EXHIBIT 2

The bill gets paid either way

With the loop
Standing stewardship, sized to how much the operation relies on the system.

Without it
Evaluation cost paid at incident prices: the degraded quarter nobody measured.

The audit answer
Due-diligence questionnaires answered from records, not meetings.

The staleness check
Last evidence date versus last change date. No gap, ever.

The cost objection — that living evaluation is overhead a lean team cannot carry — inverts the comparison. The alternative to the loop is evaluation cost paid irregularly, at incident prices, with interest: the degraded quarter nobody measured, the trust collapse after the visible failure, the re-validation scramble when an auditor asks which evidence covers the current system. The loop amortizes a bill the operation pays either way; how much it relies sets the size.

From launch gate to operating control

The move is from the closed question "did it pass?" to the standing question "what does the current evidence cover, and does our reliance still fit inside it?" The first gets answered once and ages. The second has to be answered again every time the system, the data, or the work changes — which is what the seven disciplines are for.

The habit the loop builds is the one that transfers: treating "the system works" as a claim with a date on it. The loop is the machinery that keeps the date current.

Workflows that run with this machinery around them keep changing — models update, case mixes shift — and the evaluation changes with them. The number that matters is the one that stops recurring: weeks of silent degradation. The loop never makes the system better. It makes the company aware, every week, of exactly what the system is.

Notes and sources

[1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, July 26, 2024. https://doi.org/10.6028/NIST.AI.600-1

[2] Blake Bullwinkel, Amanda Minnich, Shiven Chawla, Gary Lopez, Martin Pouliot, Whitney Maxwell, Joris de Gruyter, Katherine Pratt, Saphir Qi, Nina Chikanov, Roman Lutz, Raja Sekhar Rao Dheekonda, Bolor-Erdene Jagdagdorj, Eugenia Kim, Justin Song, Keegan Hines, Daniel Jones, Giorgio Severi, Richard Lundeen, Sam Vaughan, Victoria Westerhoff, Pete Bryan, Ram Shankar Siva Kumar, Yonatan Zunger, Chang Kawaguchi, and Mark Russinovich, "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/

[3] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015). https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023. https://arxiv.org/abs/2202.03629

[5] Raja Parasuraman and Dietrich H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," Human Factors 52, no. 3 (2010): 381-410. https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-addd42e249d2/download

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani